laboratoria 14

Testowanie akceptacyjne realizacji przypadków użycia

Cel zajęć: Weryfikacja poprawności działania tworzonego oprogramowania przy pomocy testów akceptacyjnych z użyciem frameworku *FitNesse*.

Źródła wiedzy: Wykład IO: *Testowanie oprogramowania* (dostępny też w *ePortalu Pwr*).
Przykład: *Miniprojekt Biblioteka* (dostępny też w *ePortalu PWr*).
Tutorial do FitNesse: *FitNesse User Guide*

Zawartość instrukcji: 3 zadania – ich wykonanie jest oceniane.
Spis rzeczy, które należy umieścić w sprawozdaniu z wykonania zadań.
Spis błędów i braków mogących obniżyć ocenę.

Zadania

Cel zadań:

Celem zadań jest wykonanie systemowych testów akceptacyjnych, czyli testów całego oprogramowania wykonanego w poprzednich etapach laboratoriów, w zakresie realizacji zaimplementowanych wcześniej przypadków użycia. Zwrot *całe oprogramowanie* oznacza tu, że nie można testować tylko jednej klasy lub tylko jednej warstwy w izolacji od innych, ale wszystkie powiązane ze sobą klasy i warstwy oprogramowania.

Każda osoba z zespołu wykonującego zadania powinna przetestować realizację jednego przypadku użycia.

Jeśli w poprzednich etapach laboratoriów oprogramowanie zostało wykonane prawidłowo, to realizacja przypadku użycia jest zadaniem głównej operacji (lub konstruktora) w tej klasie z warstwy kontroli, która odpowiada za ten przypadek użycia. Celem testu jest więc wywołanie tej operacji z określonymi parametrami wejściowymi (jeśli ma parametry), w odpowiednim początkowym stanie oprogramowania i sprawdzenie, czy uzyskano spodziewany wynik, czyli określony stan warstwy encji (stan danych oprogramowania).

Testy należy wykonać w wybranym środowisku IDE przy pomocy frameworku *FitNesse*. Jeśli oprogramowanie wykonano w języku innym niż Java i z jakiegoś powodu nie można użyć *FitNesse*, należy użyć innego, podobnego narzędzia.

Akceptacyjny test systemu:

Sprawdza, czy dany przypadek użycia oprogramowania działa zgodnie z oczekiwaniami wyrażonymi w opisie tego przypadku użycia i w związanych z nim wymaganiach, na poziomie całego oprogramowania (systemu) – wszystkich jego warstw razem.

Test akceptacyjny zwykle polega na utworzeniu, powiązaniu ze sobą i uruchomieniu wszystkich części testowanego oprogramowania i wykonaniu testowanego przypadku użycia.

Rzeczywisty wynik testu porównuje się z oczekiwanym wynikiem w celu określenia jakości realizacji testowanego przypadku użycia, a nie w celu wykrycia błędów implementacji lub integracji.

Zadanie 1

Weryfikacja poprawności działania tworzonego oprogramowania przy pomocy testów akceptacyjnych z użyciem frameworku *FitNesse* – analiza i przygotowanie wykonanego oprogramowania do testów.

Należy upewnić się, że kod testowanego oprogramowania umożliwia nie tylko wywołanie testowanej operacji realizacji przypadku użycia, ale też sprawdzenie stanu warstwy encji (modelu) oprogramowania przed wykonaniem i po wykonaniu testowanej operacji.

To sprawdzenie powinno być możliwe dzięki wywołaniu w warstwie encji (w klasie zarządzającej danymi przetwarzanymi przez testowany przypadek użycia) operacji, która zwraca stan warstwy encji, czyli stan danych, na które wpływ ma realizacja testowanego przypadku użycia. Np. do testu dodania obiektu *klient* do listy klientów, mogą się przydać operacje: sprawdzenie liczby klientów w tej liście i sprawdzenie stanu (wartości atrybutów) klienta ostatnio dodanego do tej listy.

Jeśli brak takich operacji, to należy je dodać do odpowiednich klas testowanego oprogramowania.

Zadanie 2

Weryfikacja poprawności działania tworzonego oprogramowania przy pomocy testów akceptacyjnych z użyciem frameworku *FitNesse* – klasa *SetUp* i klasy testujące.

Do projektu testowanego oprogramowania należy dodać bibliotekę *fitnesse-standalone.jar* oraz pakiet na klasy testujące, np. *testyfitnesse,* i umieścić w nim klasy opisane niżej – będą wykorzystywane przez framework *FitNesse*.

Najpierw należy utworzyć klasę *SetUp* (rozszerzenie klasy *Fixture*) i w niej:

– zadeklarować statyczne instancje klas realizujących testowane przypadki użycia i instancje klas udostępniających stan warstwy encji; operacje tych instancji, czyli operacje realizacji przypadku użycia i operacje zwracające stan danych związanych z realizowanym przypadkiem użycia, będą wywoływane w ciele testów;

– zaimplementować konstruktor *SetUp()*, w którym powinny być utworzone, przypisane powyższym deklaracjom i połączone ze sobą wszystkie części testowanego oprogramowania, czyli obiekty i klasy, współpracujące ze sobą w realizacji testowanych przypadków użycia.

Następnie dla testowanego przypadku użycia należy utworzyć klasę testującą realizację przypadku użycia (rozszerzenie klasy *ColumnFixture*) i w niej:

– zadeklarować atrybuty danych wejściowych testu i niczego im nie przypisywać – ich zawartość zostanie pobrana przez framework *FitNesse* ze strony testu,

– zaimplementować operację testującą realizację przypadku użycia,

– zaimplementować operacje sprawdzające stan warstwy encji, czyli stan danych, na które wpływa realizacja przypadku użycia.

Klasa *SetUp()*:

Klasa *SetUp* to klasa typu *fixture*, czyli taka, która przygotowuje przedmiot testów – testowane oprogramowanie.

Ta klasa deklaruje statyczne atrybuty na obiekty używane przez operacje testujące. Następnie jej konstruktor *SetUp()* tworzy dla tych atrybutów instancje tych klas testowanego oprogramowania, których operacje mają być testowane lub użyte przez wywołanie ich w klasach je testujących.

Klasa testująca realizację przypadku użycia:

Klasa testująca realizację przypadku użycia może zawierać kilka operacji: operację testującą operację realizacji przypadku użycia i pomocnicze operacje sprawdzające stan warstwy encji, czyli stan danych przed, a zwłaszcza po wywołaniu testowanej operacji.

Operacja testująca operację realizacji przypadku użycia:

Przykładowa zawartość ciała operacji testującej wykonanie operacji przypadku użycia (na przykładzie testu operacji *DodajK(Klasa k)* obiektu *PU*, realizującej przypadek użycia skutkujący dodaniem obiektu *k* klasy *Klasa* do listy *listaK* takich obiektów w klasie *Inwentarz* w warstwie encji):

1) Zapisanie początkowego stanu warstwy encji przez wykonanie operacji sprawdzającej ten stan,
   - przykład – wywołanie operacji *stanK()*, która zwraca rozmiar listy *listaK* i zapisanie tego rozmiaru:
   ```
   int stanPrzed = stanK();          // stanK() jest operacją tej samej klasy testującej
   ```
2) Wykonanie testowanej operacji realizującej przypadek użycia z określonymi parametrami wejściowymi,
   - przykład – wywołanie operacji *DodajK(k)* w celu dodania obiektu *k* do listy *listaK*:
   ```
   SetUp.PU.DodajK(k);               // PU jest statycznym atrybutem klasy SetUp
   ```
3) Zapisanie końcowego stanu warstwy encji przez wykonanie sprawdzającej ten stan,
   - przykład – wywołanie operacji *stanK()*:
   ```
   int stanPo = stanK();
   ```
4) Zwrócenie wyniku lub skutku wykonania testowanej operacji, np. tego, czy stan warstwy encji się zmienił,
   - przykład – zwrócenie *true*, jeśli zmienił się rozmiar listy *listaK*:
   ```
   return stanPrzed != stanPo;
   ```

Operacja sprawdzająca stan warstwy encji:

Przykładowa zawartość ciała operacji sprawdzającej stan warstwy encji (na tym samym przykładzie, co wyżej):

- operacja *stanK()* zwraca wynik operacji *ileK()* klasy *Inwentarz*, czyli rozmiaru listy *listaK*:
  ```
  return SetUp.Inwentarz.ileK();     // Inwentarz jest statycznym atrybutem klasy SetUp
  ```

Zadanie 3

Weryfikacja poprawności działania tworzonego oprogramowania przy pomocy testów akceptacyjnych z użyciem frameworku *FitNesse* – strony testujące.

Należy uruchomić usługę *FitNesse* w przeglądarce internetowej, a następnie:

1) Utworzyć stronę zestawu testów (*Suite Page*) przypadków użycia i dodać ją do strony tytułowej (*FrontPage*).
2) Utworzyć stronę *SetUp* (*Static Page*), przygotowującą testowane oprogramowanie, na stronie tego zestawu.
3) Dla każdego testu przypadku użycia utworzyć stronę testującą przypadek użycia (*Test Page*), na stronie tego zestawu.
4) Wykonać testy tego zestawu testów.

Strona zestawu testów powinna zawierać:

– nazwę określającą jej testy,
– dłuższy słowny opis tego zestawu,
– ścieżkę do pakietu (lub ścieżki do pakietów) z plikami wynikowymi (*.class*) klas testowanych i klas testujących.

Strona *SetUp* powinna zawierać:

– nazwę *SetUp*,
– ścieżkę do pliku źródłowego klasy *SetUp*.

Strona testująca przypadek użycia powinna zawierać:

– nazwę testu zawierającą w sobie nazwę testowanego przypadku użycia,
– ścieżkę do pliku źródłowego klasy testującej ten przypadek użycia,
– tabelę z danymi wejściowymi (parametry do operacji zdefiniowanych w klasie testującej) i danymi wyjściowymi (spodziewane wyniki wykonania operacji zdefiniowanych w klasie testującej); te dane wejściowe i wyjściowe powinny uwzględniać zarówno poprawne, jak i niepoprawne scenariusze wykonania testowanego przypadku użycia.

Uruchomienie usługi *FitNesse*:

Najpierw należy w terminalu wpisać polecenie uruchamiające usługę *FitNesse* na *localhost* w porcie 8080 (numer portu może być inny, byle był wolny): `java -jar fitnesse-standalone.jar -p 8080`

Następnie należy uruchomić tę usługę w przeglądarce internetowej pod adresem `http://localhost:8080`

Utworzenie strony zestawu testów:

Najpierw należy utworzyć stronę zestawu testów (menu *Add* → *Suite Page*) i na stronie, która się pokaże:

– w polu tekstowym *Page name* wpisać nazwę tej strony, np. *Biblioteka* (nazwa musi mieć choć jedną dużą literę i nie może zawierać słowa *test*);
– w edytorze tekstowym na dole strony dopisać do tego, co już jest:
  • słowny opis zestawu testów, np. *"Testy akceptacyjne systemu zarządzania biblioteką"*
  • ścieżkę lub ścieżki do pakietów z plikami wynikowymi (*.class*) klas testowanych i klas testujących, np.

```
!path /home/autor/Projekty/Nazwa Projektu/bin/
```

Następnie należy dopisać link do tej strony do treści strony tytułowej (lub zamiast niej), czyli wejść na stronę `http://localhost:8080/FrontPage` i w jej edycję (menu *Edit*) i na stronie, która się pokaże, w edytorze tekstowym na dole strony wpisać link do utworzonej strony zestawu testów, np. `[[NazwaStronyZestawuTestów][FrontPage.NazwaStronyZestawuTestów]]`

Edycję strony zatwierdza kliknięcie przycisku *Save.*

Utworzenie strony *SetUp*:

Najpierw należy wejść na utworzoną wcześniej stronę zestawu testów, a następnie utworzyć stronę *SetUp* (menu *Add* → *Static Page*) i na stronie, która się pokaże:

– w polu tekstowym *Page name* wpisać nazwę tej strony *SetUp*;
– w edytorze tekstowym na dole strony wpisać ścieżkę do klasy *SetUp*, np. `!|testyfitnesse.SetUp|`

Edycję strony zatwierdza kliknięcie przycisku *Save.*

Utworzenie strony testującej:

Najpierw należy wejść na utworzoną wcześniej stronę zestawu testów, a następnie utworzyć stronę testującą dany przypadek użycia (menu *Add* → *Test Page*) i na stronie, która się pokaże:

– w polu tekstowym *Page name* wpisać nazwę tej strony, np. *WypisanieCzytelnikaZBiblioteki*;

– w edytorze tekstowym na dole strony wpisać ścieżkę do klasy testującej, np.

```
!|testyfitnesse.testWypisaniaCzytelnika|
```

– niżej w tym edytorze wpisać nagłówki tabeli danych wejściowych i wyjściowych testu, np.

```
|nrCzytelnika|wypiszCzytelnikaZBiblioteki?|dajLliczbęCzytelników?|
```

  - jeśli kolumna zawiera dane wejściowe – wartości atrybutu testującej klasy, to należy wpisać jego nazwę,
  - jeśli kolumna zawiera dane wyjściowe – asercje wyniku wykonania operacji testującej klasy, to należy wpisać jej nazwę i pytajnik,

– niżej w tym edytorze wpisać wartości danych wejściowych i wyjściowych testu, np.

```
|1           |true                    |5                    |
|2           |true                    |4                    |
|2           |false                   |4                    |
|7           |false                   |4                    |
```

  - każdy wiersz danych posłuży do osobnego wykonania testu – osobno dla danych tego wiersza.

Edycję strony zatwierdza kliknięcie przycisku *Save.*

Wykonanie testów:

W celu wykonania pojedynczego testu, należy wejść na jego stronę testującą i wybrać menu *Test*.
W celu wykonania całego zestawu testów, należy wejść na stronę tego zestawu i wybrać menu *Suite*.

Zadania opracowano na podstawie instrukcji 10 „Laboratorium z przedmiotu: Inżynieria Oprogramowania W04ITE-SI0011G”, autorstwa dr inż.

Sprawozdanie

Co powinno być na początku sprawozdania:

– Autorzy (imię, nazwisko, nr albumu).
– Nazwa tworzonego oprogramowania (zwięzły opis jego dziedziny).
– Temat tego etapu laboratoriów (*Testowanie akceptacyjne realizacji przypadków użycia*).

Co powinno być w treści sprawozdania:

– Lista zmian w kodzie źródłowym testowanego oprogramowania, zrobionych w celu wykonania testów.
– Pełny kod klasy SetUp i wszystkich klas testujących (tekst, a nie obraz ze zdjęciem ekranu), zawierający:
  - klasy w kolejności ich w pełni kwalifikowanych nazw (czyli wraz ze ścieżką pakietów, np. pakiet.klasa.java),
  - nazwę pliku z definicją klasy (nad tą definicją),
  - całą zawartość tego pliku,
  - numerację linii,
  - komentarze do mało oczywistych rzeczy,
  - wcięcia i inne formatowania ułatwiające czytelność.
– Pełną treść strony *SetUp* (tekst, a nie obraz ze zdjęciem ekranu), zawierającą
  - ścieżkę do klasy *SetUp*.
– Nazwę i pełną treść każdej strony testującej (tekst, a nie obraz ze zdjęciem ekranu), zawierającą:
  - ścieżkę do klasy testującej,
  - tabelę danych wejściowych i wyjściowych testu.
– Wynik wykonania zestawu testu (obraz ze zdjęciem ekranu przeglądarki) z kolorowymi komórkami tabeli.

Błędy i braki

Jeśli opisane niżej rzeczy mają miejsce, ocena za ten etap laboratoriów może być niższa:

– Konstruktor klasy *SetUp* nie tworzy całego testowanego oprogramowania, lub przynajmniej tej części, która wykonuje testowane przypadki użycia, ale np. stosuje atrapy itp. symulacje.
– Testowana operacja nie jest operacją realizacji przypadku użycia.
– Testowana operacja realizacji przypadku użycia powoduje zrealizowanie tylko cześci tego przypadku użycia.
– Nazwa klasy testującej nie zawiera nazwy testowanego przypadku użycia.
– Kod klas nie jest sformatowany lub jest niepraktycznie sformatowany, np. brak komentarzy, wcięć czy numeracji linii.

– Tabela danych na stronie testującej nie zawiera różnych możliwych (pożądaych i niepożądanych) wyników testowanej operacji dla różnych wartości jej parametrów wejściowych, chociaż wynik realizacji przypadku użycia zależy od tych parametrów.
– Brak strony zestawu testów.
– Strona *SetUp* nie jest włączona do zestawu testów.
– Strona testująca nie jest włączona do zestawu testów.

Ta lista zawiera typowo pojawiające się błędy i braki i nie wyczerpuje powodów, dla których ocena za laboratoria może być niższa.